

IT3042 – LẬP TRÌNH BACKEND LARAVEL

BUỔI 7: ELOQUENT RELATIONSHIPS

Quan hệ giữa các Model trong Laravel

Khoa Công nghệ Thông tin · Đại học Phú Xuân · 3 tín chỉ

Thông tin	Chi tiết
Học phần	IT3042 – Lập trình Backend Laravel
Buổi học	Buổi 7 / 15 (Giai đoạn 2: Database & Eloquent)
Thời lượng	180 phút (lý thuyết + thực hành xen kẽ)
Dự án thực hành	phu-xuan-blog
Công cụ yêu cầu	Laravel 10, PHP 8.1+, Laravel Debugbar, TablePlus

I. MỤC TIÊU BÀI GIẢNG

Chuẩn đầu ra (Bloom's Taxonomy)
 NHỚ – Liệt kê 5 loại quan hệ Eloquent: hasOne, hasMany, belongsTo, belongsToMany, hasManyThrough
 HIỂU – Giải thích cơ chế foreign key, pivot table và cách Eloquent ánh xạ chúng thành object PHP
 ÁP DỤNG – Khai báo và sử dụng relationship đúng cách trong Model, Controller, View của blog app
 PHÂN TÍCH – Phân biệt Lazy Loading vs Eager Loading, nhận diện N+1 query problem bằng Debugbar
 TỔNG HỢP – Xây dựng module Post–Category–Tag–Comment có đầy đủ quan hệ trong phu-xuan-blog

II. BỐI CẢNH & ÔN TẬP NHANH

Đã được học Eloquent ORM cơ bản: tạo Model, \$fillable, CRUD với create()/save()/update()/delete(). Sang Buổi 7, chúng ta đi sâu vào phần quan trọng nhất của Eloquent — cách các Model "nói chuyện" với nhau thông qua Relationships.

Câu hỏi ôn tập nhanh (5 phút)
1. Model Post cần có thuộc tính gì để Eloquent cho phép gán dữ liệu hàng loạt?
2. Lệnh nào tạo Model kèm Migration cùng lúc?
3. Sự khác nhau giữa Post::all() và Post::where("status","published")->get()?

III. NỘI DUNG LÝ THUYẾT

3.1 Tổng quan về Eloquent Relationships

Eloquent Relationships là tính năng cho phép bạn định nghĩa và truy vấn mối liên kết giữa các bảng dữ liệu thông qua các phương thức PHP, thay vì viết SQL JOIN thủ công.

Các loại quan hệ trong Eloquent:

Phương thức	Ý nghĩa	Ví dụ thực tế
hasOne()	Một-một (1:1)	User → UserProfile
hasMany()	Một-nhiều (1:N)	Post → Comment[]
belongsTo()	Ngược của hasOne/hasMany	Comment → Post
belongsToMany()	Nhiều-nhiều (N:N)	Post ↔ Tag[]
hasManyThrough()	Quan hệ qua bảng trung gian	Country → Post qua User
morphTo / morphMany	Đa hình (polymorphic)	Image thuộc Post hoặc User

3.2 hasOne — Quan hệ Một-Một

Dùng khi một Model chỉ liên kết với đúng một bản ghi của Model khác.

Ví dụ: Mỗi User có đúng một UserProfile.

Cấu trúc database:

```
-- users table
id | name | email | ...

-- user_profiles table
id | user_id (FK) | bio | avatar | social_links | ...
```

Khai báo trong Model:

```
// app/Models/User.php
class User extends Model {
    public function profile(): HasOne
    {
        return $this->hasOne(UserProfile::class);
        // Eloquent tự suy: foreign key = "user_id" trên bảng user_profiles
        // Nếu khác: hasOne(UserProfile::class, "foreign_key", "local_key")
    }
}

// app/Models/UserProfile.php
class UserProfile extends Model {
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Sử dụng:

```
$user = User::find(1);

// Truy cập profile (lazy load - chạy thêm 1 SQL)
$bio = $user->profile->bio;

// Tạo profile cho user
$user->profile()->create([
    'bio'      => 'Giảng viên CNTT',
    'avatar'   => 'avatar.jpg',
]);

// Cập nhật
$user->profile()->update(["bio" => "Cập nhật bio"]);
```

3.3 hasMany & belongsTo — Quan hệ Một-Nhiều

Đây là quan hệ phổ biến nhất. Một Post có nhiều Comment; mỗi Comment thuộc về một Post.

Cấu trúc database:

```
-- posts table
id | user_id | category_id | title | body | status | published_at

-- comments table
id | post_id (FK) | user_id (FK) | body | is_approved | created_at
```

Khai báo trong Model:

```
// app/Models/Post.php
class Post extends Model {
    // Post có nhiều Comment
    public function comments(): HasMany
    {
        return $this->hasMany(Comment::class);
        // FK mặc định: "post_id" trên bảng comments
    }

    // Post thuộc về một User (tác giả)
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    // Post thuộc về một Category
    public function category(): BelongsTo
    {
        return $this->belongsTo(Category::class);
    }
}

// app/Models/Comment.php
class Comment extends Model {
    public function post(): BelongsTo
    {
        return $this->belongsTo(Post::class);
    }
}
```

```

    }
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}

```

Các thao tác thực tế:

```

$post = Post::find(1);

// Lấy tất cả comment của post
$comments = $post->comments; // Collection
$approved = $post->comments()->where("is_approved", true)->get();

// Đếm số comment
$count = $post->comments()->count();

// Tạo comment cho post (tự điền post_id)
$post->comments()->create([
    'user_id' => auth()->id(),
    'body'    => 'Comment hay!',
]);

// Xóa tất cả comment (dùng khi xóa post)
$post->comments()->delete();

```

3.4 belongsToMany — Quan hệ Nhiều-Nhiều

Dùng khi cả hai phía đều có thể liên kết với nhiều bản ghi của phía kia. Ví dụ: một Post có nhiều Tag, một Tag gắn với nhiều Post.

Cần 3 bảng:

```

-- posts      : id, title, ...
-- tags       : id, name, slug
-- post_tag   : post_id (FK), tag_id (FK) ← pivot table

-- Lưu ý đặt tên pivot: 2 tên bảng, số ít, thứ tự alphabet
-- "post" trước "tag" → bảng tên "post_tag"

```

Tạo migration cho pivot table:

```

// database/migrations/..._create_post_tag_table.php
Schema::create("post_tag", function (Blueprint $table) {
    $table->foreignId("post_id")->constrained()->cascadeOnDelete();
    $table->foreignId("tag_id")->constrained()->cascadeOnDelete();
    $table->primary(["post_id", "tag_id"]); // composite PK
});

```

Khai báo trong Model:

```

// app/Models/Post.php
public function tags(): BelongsToMany

```

```
{
    return $this->belongsToMany(Tag::class);
    // Eloquent tự suy: pivot = "post_tag"
    // FK lần lượt: "post_id", "tag_id"
}

// app/Models/Tag.php
public function posts(): BelongsToMany
{
    return $this->belongsToMany(Post::class);
}
```

Các thao tác trên pivot:

```
$post = Post::find(1);

// Gắn tag (thêm vào pivot)
$post->tags()->attach([1, 2, 3]);
$post->tags()->attach($tagId, ["order" => 1]); // kèm extra column

// Gỡ tag (xóa khỏi pivot)
$post->tags()->detach([2, 3]);
$post->tags()->detach(); // gỡ tất cả

// Sync: chỉ giữ đúng danh sách tag_id này
$post->tags()->sync([1, 4, 5]); // tự add/remove để khớp
$post->tags()->syncWithoutDetaching([6]); // chỉ add, không remove

// Truy cập dữ liệu pivot (khai báo withPivot() trước)
$post->tags()->withPivot("order")->get()->each(function($tag) {
    echo $tag->pivot->order;
});
```

3.5 N+1 Problem & Eager Loading

Đây là chủ đề cực kỳ quan trọng về hiệu năng mà mọi developer Laravel cần nắm vững.

N+1 Problem là gì?

Khi truy vấn N bài viết và access relationship từng cái một, Eloquent chạy:

- 1 query để lấy danh sách Post
- N query để lấy user của từng Post (mỗi post 1 query)

→ Tổng: 1 + N queries. Với N = 100 posts → 101 SQL queries!

Đây là anti-pattern phổ biến nhất, gây chậm ứng dụng nghiêm trọng.

Ví dụ gây ra N+1:

```
// ❌ BAD – N+1 problem
$post = Post::all();           // Query 1: SELECT * FROM posts

foreach ($posts as $post) {
    echo $post->user->name;      // Query 2,3,4,...N+1
    // Mỗi lần access $post->user, Eloquent chạy thêm 1 SQL:
    // SELECT * FROM users WHERE id = ?
}
```

Giải quyết bằng Eager Loading:

```
// ✅ GOOD – Eager Loading với with()
$post = Post::with("user")->get();
// Chỉ 2 queries:
// Query 1: SELECT * FROM posts
// Query 2: SELECT * FROM users WHERE id IN (1,2,3,...)

foreach ($posts as $post) {
    echo $post->user->name;      // Không thêm query nào!
}

// Load nhiều relationship cùng lúc
$post = Post::with(["user", "category", "tags", "comments"])->get();

// Nested eager loading (quan hệ lồng nhau)
$post = Post::with("comments.user")->get();
// Lấy post, kèm comments, kèm user của từng comment

// Eager loading có điều kiện
$post = Post::with(["comments" => function($query) {
    $query->where("is_approved", true)->latest();
}])->get();
```

Lazy Eager Loading (load sau khi đã lấy):

```
$post = Post::find(1);

// Chưa load relationship, load thêm sau
$post->load("comments");
```

```
$post->load(["comments", "tags"]);

// Dùng khi không biết trước có cần load không
if ($needComments) {
    $post->load("comments");
}
```

Bảng so sánh Lazy vs Eager Loading:

Tiêu chí	Lazy Loading	Eager Loading
Cú pháp	\$post->user (access trực tiếp)	Post::with("user")->get()
Số queries	1 + N (nguy hiểm!)	2 (luôn cố định)
Khi nào dùng	Single model, 1-2 field	Danh sách, loop, API
Detect N+1	Laravel Debugbar đỏ	Debugbar chỉ 2 queries
Khuyến nghị	Tránh trong vòng lặp	Luôn dùng cho list

3.6 withCount(), has() và whereHas()

Eloquent cung cấp các phương thức mạnh mẽ để truy vấn thông qua relationship mà không cần tải toàn bộ dữ liệu.

```
// withCount() - đếm số bản ghi liên quan (thêm cột {relation}_count)
$posts = Post::withCount("comments")->get();
foreach ($posts as $post) {
    echo $post->comments_count; // không cần lấy toàn bộ comments!
}

// withCount nhiều relationship + alias
$posts = Post::withCount([
    "comments",
    "tags",
    "comments as approved_count" => function($q) {
        $q->where("is_approved", true);
    }
])->get();

// has() - lọc chỉ lấy Post có ít nhất 1 comment
$postsWithComments = Post::has("comments")->get();
$postsWithTags = Post::has("tags", ">=", 2)->get(); // có ít nhất 2 tag

// whereHas() - lọc theo điều kiện trong relationship
$posts = Post::whereHas("comments", function($query) {
    $query->where("is_approved", true);
})->get();
// = SELECT posts WHERE EXISTS (SELECT 1 FROM comments ...)

// doesntHave() - Post chưa có comment nào
$orphanPosts = Post::doesntHave("comments")->get();
```

3.7 hasManyThrough — Quan hệ qua bảng trung gian

Cho phép truy cập quan hệ "xa" thông qua model trung gian. Ví dụ: Category → (thông qua Post) → Comment.

```
// Category có nhiều Comment (thông qua Post)
// categories → posts → comments

// app/Models/Category.php
public function comments(): HasManyThrough
{
    return $this->hasManyThrough(
        Comment::class, // Model cuối
        Post::class,    // Model trung gian
        "category_id",  // FK trên bảng posts → categories
        "post_id",      // FK trên bảng comments → posts
        "id",            // Local key trên categories
        "id"             // Local key trên posts
    );
}

// Sử dụng
$category = Category::find(1);
$allComments = $category->comments()->latest()->take(10)->get();
```

3.8 Polymorphic Relationships — Quan hệ đa hình

Cho phép một Model liên kết với nhiều loại Model khác nhau qua một bảng chung.

Ví dụ: Bảng images có thể lưu ảnh của cả Post lẫn User.

```
-- images table
id | imageable_id | imageable_type | url
1  | 5            | App\Models\Post | post-cover.jpg
2  | 12           | App\Models\User | avatar.jpg

-- imageable_type lưu class name của Model liên quan
-- imageable_id   lưu id của record tương ứng
```

```
// app/Models/Image.php
class Image extends Model {
    public function imageable(): MorphTo
    {
        return $this->morphTo();
    }
}

// app/Models/Post.php
public function images(): MorphMany
{
    return $this->morphMany(Image::class, "imageable");
}

// app/Models/User.php
public function images(): MorphMany
{
    return $this->morphMany(Image::class, "imageable");
}

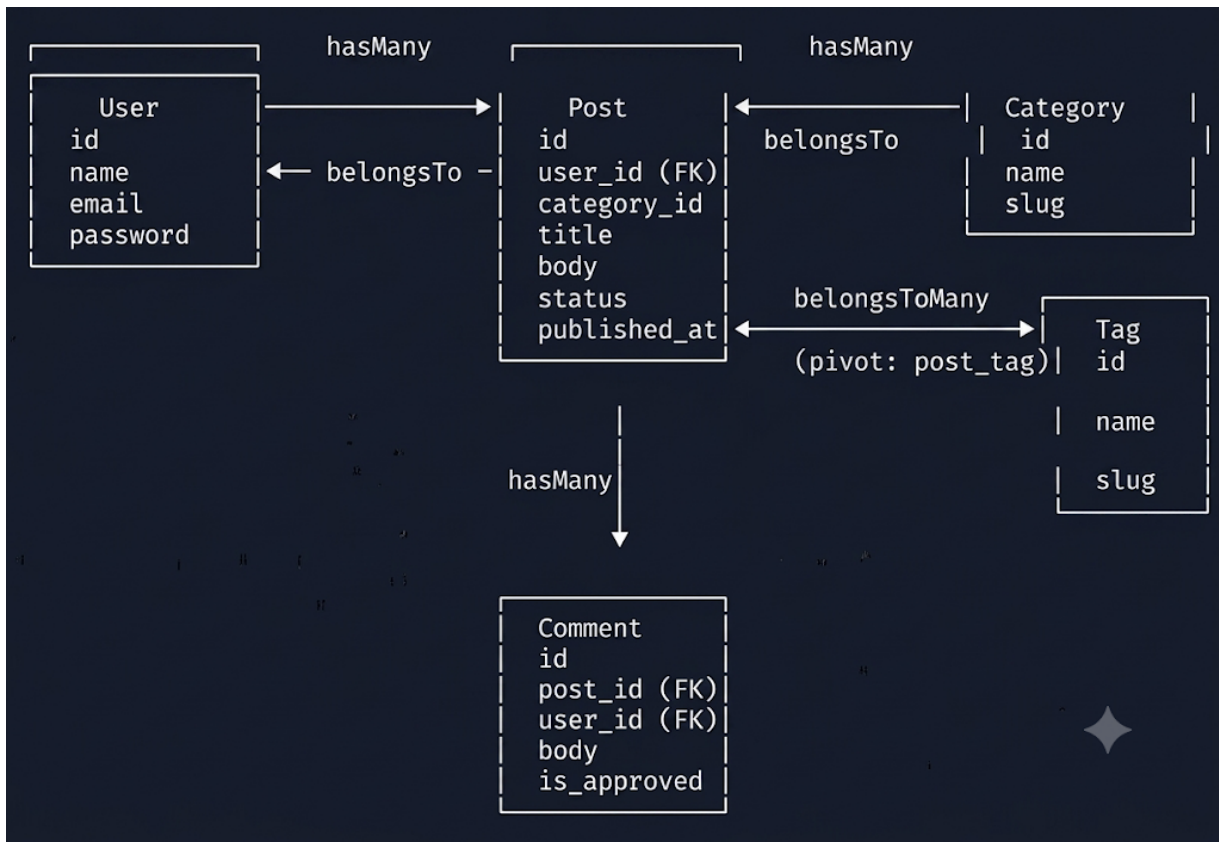
// Sử dụng
$post->images()->create(["url" => "cover.jpg"]);
```



```
$image = Image::find(1);  
$owner = $image->imageable; // Trả về Post hoặc User
```

IV. SƠ ĐỒ QUAN HỆ – PHU-XUAN-BLOG

Tổng hợp toàn bộ quan hệ trong dự án thực hành:



Tóm tắt quan hệ trong blog app

User hasMany Post (1 user viết nhiều bài)
 User hasMany Comment (1 user đăng nhiều comment)
 Post belongsTo User (bài viết có tác giả)
 Post belongsTo Category
 Post hasMany Comment
 Post belongsToMany Tag (qua pivot post_tag)
 Tag belongsToMany Post
 Comment belongsTo Post
 Comment belongsTo User

V. PHÂN TÍCH CHUYÊN SÂU

5.1 Convention vs Configuration

Eloquent theo nguyên tắc "Convention over Configuration" – nếu đặt tên đúng quy ước, không cần cấu hình thêm.

Quy ước	Ví dụ	Override khi nào
FK = tên model + id	post id, category id	belongsTo(Post::class, "article id")
Pivot = 2 tên model alphabet	post tag, category post	belongsToMany(Tag::class, "taggables")
Local key = id	posts.id	hasMany(Comment::class, "post_id", "uuid")
Table = tên model số nhiều	posts, categories, users	protected \$table = "blog_posts"

5.2 Khi nào dùng loại quan hệ nào?

Quy tắc đơn giản để chọn đúng loại relationship:

◆ Đặt câu hỏi: "A có/thuộc bao nhiêu B?"

- "1 User có 1 Profile" →hasOne / belongsTo
- "1 Post có nhiều Comment" →hasMany / belongsTo
- "1 Post có nhiều Tag, 1 Tag ở nhiều Post" → belongsToMany
- "Category có Comments qua Post" →hasManyThrough
- "Image có thể của Post hoặc User" →polymorphic (morphMany/morphTo)

5.3 Chiến lược tối ưu hiệu năng

Nguyên tắc 4 bước khi làm việc với Relationship:

1. Luôn dùng Eager Loading (with()) khi render danh sách có relationship
2. Dùng withCount() thay vì load collection chỉ để đếm
3. Chỉ select() các cột cần thiết, không SELECT * khi dữ liệu lớn
4. Cài Laravel Debugbar và kiểm tra số query mỗi request

```
// Tối ưu hóa query khi render trang danh sách Post
$post = Post::select("id", "title", "user_id", "category_id", "created_at")
->with([
    "user:id,name", // chỉ lấy id và name của user
    "category:id,name,slug", // chỉ lấy 3 cột category
    "tags:id,name", // chỉ lấy id và name tag
])
->withCount("comments")
->where("status", "published")
->latest("published_at")
->paginate(10);

// Kết quả: CHỈ 4 queries dù hiển thị 10 posts với user, category, tags,
comments_count
```

VI. KẾ HOẠCH GIẢNG DẠY 180 PHÚT

VII. HƯỚNG DẪN THỰC HÀNH

Lab 1: Thiết lập quan hệ Post–Comment–User (35 phút)

Bước 1: Tạo Migration cho bảng comments

```
php artisan make:migration create_comments_table

// Schema::create("comments", function (Blueprint $table) {
    $table->id();
    $table->foreignId("post_id")->constrained()->cascadeOnDelete();
    $table->foreignId("user_id")->constrained()->cascadeOnDelete();
    $table->text("body");
    $table->boolean("is_approved")->default(false);
    $table->timestamps();
// });
```

Bước 2: Tạo Model Comment

```
php artisan make:model Comment

// app/Models/Comment.php
class Comment extends Model {
    protected $fillable = ["post_id", "user_id", "body", "is_approved"];

    public function post(): BelongsTo {
        return $this->belongsTo(Post::class);
    }
    public function user(): BelongsTo {
        return $this->belongsTo(User::class);
    }
}
```

Bước 3: Thêm relationship vào Post Model

```
// app/Models/Post.php – thêm method:
public function comments(): HasMany {
    return $this->hasMany(Comment::class);
}
public function approvedComments(): HasMany {
    return $this->hasMany(Comment::class)->where("is_approved", true);
}
```

Bước 4: Hiện thị comment trong View

```
{{-- resources/views/posts/show.blade.php --}}
<h3>Bình luận ({{ $post->comments_count }})</h3>
@foreach($post->approvedComments as $comment)
    <div class="comment">
        <strong>{{ $comment->user->name }}</strong>
        <p>{{ $comment->body }}</p>
        <small>{{ $comment->created_at->diffForHumans() }}</small>
    </div>
```

```
@endforeach
```

Checkpoint Lab 1

- ✓ Bảng comments đã migrate thành công
- ✓ Model Comment có 2 quan hệ belongsTo
- ✓ Model Post có hasMany comments
- ✓ Trang show.blade.php hiển thị được danh sách comment
- ✓ Laravel Debugbar: trang post detail không quá 5 queries

Lab 2: Quan hệ Post–Tag (Many-to-Many) (25 phút)

Bước 1: Migration cho tags và post_tag

```
php artisan make:migration create_tags_table
php artisan make:migration create_post_tag_table

// create_tags_table
// $table->id(); $table->string("name"); $table->string("slug")->unique();

// create_post_tag_table
// $table->foreignId("post_id")->constrained()->cascadeOnDelete();
// $table->foreignId("tag_id")->constrained()->cascadeOnDelete();
// $table->primary(["post_id", "tag_id"]);
```

Bước 2: Khai báo Model Tag và cập nhật Post

```
// app/Models/Tag.php
protected $fillable = ["name", "slug"];
public function posts(): BelongsToMany {
    return $this->belongsToMany(Post::class);
}

// app/Models/Post.php – thêm:
public function tags(): BelongsToMany {
    return $this->belongsToMany(Tag::class);
}
```

Bước 3: Seed dữ liệu mẫu

```
// database/seeder/TagSeeder.php
$tags = ["Laravel", "PHP", "Backend", "API", "Tutorial"];
foreach ($tags as $name) {
    Tag::create(["name" => $name, "slug" => Str::slug($name)]);
}

// Gắn tag ngẫu nhiên cho mỗi post
Post::all()->each(function($post) {
    $post->tags()->sync(Tag::inRandomOrder()->limit(3)->pluck("id"));
});
```

Lab 3: Refactor N+1 — Tối ưu trang danh sách Post (20 phút)

Trước khi tối ưu (trong `PostController@index`):

```
// ❌ BAD – gây N+1
$post = Post::all();
return view("posts.index", compact("posts"));

// Blade:
// {{ $post->user->name }} ← mỗi cái này = 1 query thêm!
// {{ $post->category->name }}
// {{ $post->tags->count() }}
```

Sau khi tối ưu:

```
// ✅ GOOD – Eager Loading toàn diện
$post =
    Post::select("id","title","user_id","category_id","created_at","status")
        ->with([
            "user:id,name",
            "category:id,name",
            "tags:id,name",
        ])
        ->withCount("comments")
        ->where("status", "published")
        ->latest()
        ->paginate(10);

// Trong Blade, $post->comments_count thay cho $post->comments()->count()
// Kết quả: 4 queries dù có 10 posts × 3 relationship
```

Kiểm tra với Laravel Debugbar

1. Cài: `composer require barryvdh/laravel-debugbar --dev`
2. Mở trang /posts trong browser
3. Xem tab "Queries" ở thanh debug phía dưới
4. TRƯỚC tối ưu: 30–50 queries (tùy số post)
5. SAU tối ưu: chỉ 4–5 queries (1 post + 1 user + 1 category + 1 tag + 1 count)

VIII. LỖI THƯỜNG GẶP & CÁCH SỬA

Lỗi / Triệu chứng	Nguyên nhân	Cách sửa
Call to undefined method App\Models\Post::comments()	Chưa khai báo hasMany trong Model	Thêm public function comments(): HasMany vào Post.php
SQLSTATE: Column not found: user id	Migration chưa chạy hoặc thiếu FK column	php artisan migrate hoặc kiểm tra migration file
Trying to get property of non-object (null)	Relationship trả về null (bản ghi không tồn tại)	Dùng optional(\$post->user)?->name hoặc kiểm tra null trước
App\Models\Post::with() chạy nhưng vẫn N+1	Quên không load nested: with("comments") thay vì with("comments.user")	Dùng with("comments.user") để load user của comment luôn
Duplicate entry cho post_tag	attach() gọi lại khi tag đã tồn tại	Thay attach() bằng sync() hoặc syncWithoutDetaching()
pivot không có data	Chưa khai báo withPivot() trong relationship	Thêm ->withPivot("column_name") vào belongsToMany()

IX. EXIT TICKET & BÀI TẬP VỀ NHÀ

Exit Ticket (5 phút cuối buổi)

- ?** Câu 1: Viết 2 dòng code để lấy tất cả comment ĐƯỢC DUYỆT của Post id=5, eager load kèm user.
- ?** Câu 2: N+1 problem là gì? Cho ví dụ 1 dòng code gây N+1 và cách sửa.
- ?** Câu 3: Dùng method nào để gắn tag [1,2,3] vào Post, đảm bảo không bị trùng lặp?

Bài tập về nhà

- Hoàn thiện trang chi tiết Post (show.blade.php): hiển thị danh sách comment kèm tên tác giả và thời gian đăng, hiển thị tất cả tag của bài viết có link dẫn đến trang tag.
- Tạo trang /tags/{slug}: liệt kê tất cả Post có tag đó, eager load user và category, paginate 8 bài/trang.
- [Nâng cao] Viết Scope trên Post: scope("published") và scope("byTag", \$tagId) kết hợp whereHas. Thêm withCount("comments") và hiển thị trong danh sách.

Kết nối với Buổi 8

Buổi 8 sẽ học: Seeder & Factory (tạo 100 bài viết giả), Local Scope, Accessor & Mutator, và Query Builder nâng cao.
 Đây cũng là buổi Kiểm tra giữa kỳ — cần hoàn thiện blog app có đủ CRUD + relationships!

X. TÀI LIỆU THAM KHẢO

- Laravel Official Docs – Eloquent Relationships: <https://laravel.com/docs/10.x/eloquent-relationships>
- Laracasts – Eloquent Relationships Series (Jeffrey Way)

- [Laravel Daily – N+1 Problem & Solutions \(YouTube\)](#)
- [Spatie – Laravel Query Builder \(thư viện mở rộng query relationship\)](#)
- [GitHub: phu-xuan-blog – branch: buoi7-relationships](#)

— Khoa Công nghệ Thông tin · Đại học Phú Xuân —